챕터 1. 내 로컬에선 되는데. Docker로 시작하는 개발 환경

이번 챕터가 끝나면

- "내 컴퓨터에선 되는데 왜 다른 데선 안 되지"가 왜 일어나는지 직접 확인합니다 (환경 차이)
- 실행 환경을 통째로 묶어 두는 방법을 익힙니다 (컨테이너·이미지)
- `docker compose up` 한 줄로 Spring Boot 서버를 띄워 보겠습니다 (Docker Compose)

챕터 1은 기반 챕터입니다

이 책은 하나의 Spring Boot 서비스를 9단계로 성장시키는 여정입니다. 본격적인 기술(OAuth, Redis, S3, Elasticsearch, RabbitMQ)을 하나씩 얹기 전에, **모든 개발자가 똑같이 실행할 수 있는 공통 바닥**을 먼저 깔아 둡니다. 이 챕터가 끝나면 앞으로 나오는 어떤 기술을 붙이더라도, 옆자리 동료도 똑같은 한 줄 명령으로 서비스를 띄울 수 있게 됩니다.

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 준비

이 책의 실습은 GitHub 레포 두 개를 사용합니다.

레포	용도	주소
spring-app	컨테이너 안에서 돌아갈 Spring Boot 서버	`github.com/kjh5848/spring-app`
spring-docker	Dockerfile + docker-compose 구성	`github.com/kjh5848/spring-docker`

아직 클론하지 않았다면 터미널에서 실행합니다.

터미널 레포 클론

```
git clone https://github.com/kjh5848/spring-docker.git
cd spring-docker
```

파일 구조는 이렇습니다.

spring-docker 디렉토리

```
spring-docker/
├── Dockerfile              # [실습] 이미지를 굽는 레시피
├── entrypoint.sh           # [실습] 컨테이너가 켜지면 실행할 스크립트
└── docker-compose.yml      # [실습] 컨테이너를 어떻게 띄울지 정의
```

> **이미지, 컨테이너, 컴포즈 같은 단어가 낯설어도 괜찮습니다.** 파일 이름에 먼저 등장한 것이고, 각 절에서 하나씩 풀어 설명합니다.

`[실습]` 파일은 이미 채워진 상태로 준비되어 있습니다. 챕터를 따라 하며 파일을 열어 **어떤 줄이 무엇을 하는지** 이해하고, `docker compose up` 명령어로 결과를 확인하면 됩니다. 이번 챕터는 코드를 작성한다기보다, 남이 써 놓은 환경 설정을 **읽고 실행하는** 실습입니다.

2. 실습 환경 구축

Docker Desktop을 설치합니다 (Windows·macOS 공용). 설치 후 Docker Desktop 앱을 실행해서 **우측 하단 고래 아이콘이 초록색**이 되는지 확인합니다.

터미널 Docker 설치 확인

```
docker --version
docker compose version
```

`Docker version 24.x` 이상, `Docker Compose version v2.x` 이상이면 준비 완료입니다. Compose가 `docker-compose` (하이픈 구버전)가 아니라 `docker compose` (띄어쓰기, v2)로 동작하는지 반드시 확인하세요. 이 책의 모든 명령어는 v2 기준입니다.

Docker Desktop 대신 쓰고 싶다면

회사 정책으로 Docker Desktop을 못 쓰는 경우에는 **Colima** (macOS·Linux), **Rancher Desktop**, **Podman Desktop** 같은 대체 런타임도 있습니다. 이 책은 Docker Desktop을 기준으로 설명하지만, CLI(`docker`, `docker compose`)는 어느 런타임에서도 동일하게 동작합니다.

3. 사용할 구성 요소

이번 챕터는 새 라이브러리를 설치하지 않습니다. 대신 Docker가 제공하는 세 가지 재료를 씁니다.

재료	역할
베이스 이미지 (`eclipse-temurin:21-jdk`)	Java 21이 깔린 리눅스 환경
Dockerfile	이미지 위에 내 코드·도구를 얹는 레시피
docker-compose.yml	만들어진 이미지를 어떻게 띄울지 정의

지금은 개념만 잡으세요

Docker의 정식 용어(이미지·컨테이너·레이어·볼륨)가 한꺼번에 쏟아져 부담스러울 수 있습니다. 지금은 "레시피로 찍어낸 똑같은 도시락을 어디서나 꺼내 먹을 수 있다"는 **감각**만 잡으면 충분합니다. 볼륨·네트워크·멀티 스테이지 빌드 같은 세부는 뒤 챕터에서 필요할 때마다 다시 꺼내 씁니다.

4. 실습 순서

이번 챕터의 절 4개는 이 순서로 진행합니다.

1. `1.1`. 호스트에 바로 띄워 보고 실패 체험
2. `1.2`. Dockerfile · entrypoint.sh · docker-compose.yml 읽기
3. `1.3`. `docker compose up`으로 서버 띄우기
4. `1.4`. 실행 결과 확인 + `docker compose down`으로 정리

먼저 Docker 없이 서버를 띄워 보고(1.1) 문제가 무엇인지 체감한 뒤, 구성 파일 세 개를 하나씩 읽어 의미를 잡습니다(1.2). 그다음 한 줄 명령어로 서버를 띄워 똑같은 기능을 재현하고(1.3), 실행 결과를 브라우저로 확인한 뒤 정리합니다(1.4). **1.1부터 순서대로 진행하세요.**

1.1 "내 로컬에선 되는데"의 진실

그림 1-1. 첫 출근 날 아침, 서비스 저장소 URL 하나

첫 출근 날 아침.

형광등이 두 번 깜빡이고 켜지는 사무실. 키보드 소리와 커피 머신 돌아가는 소리가 번갈아 들렸습니다. 모니터 앞에 앉은 지 10분쯤 지났을 때, 팀장이 메신저로 링크 하나를 보냈습니다.

팀장 : "일단 이거 클론해서 띄워 봐요. 오픈이 씨 자리에서 `hello world!`까지 뜨면 환경 세팅 끝이에요."

링크는 Spring Boot 서비스 저장소였습니다. 간단한 `/` 엔드포인트 하나, 딱 그뿐이었습니다.

이 정도면 점심 전에 끝나겠네.

클론을 받았습니다. IntelliJ를 열고 Gradle 동기화를 걸었습니다. 빨간 줄이 떴습니다. 처음엔 어디서 본 듯한 메시지였습니다.

```
Unsupported class file major version 65
```

아.

손에 쥐고 있던 커피 컵을 내려놓았습니다. 이 회사는 Java 21을 쓴다고 공지에 적혀 있었는데, 오픈이의 노트북에는 예전 프로젝트에서 쓰던 Java 17이 남아 있었습니다. JDK 21을 새로 내려받아 `JAVA_HOME`을 바꿨습니다. 이번엔 Gradle이 다른 말을 했습니다.

```
> Task :compileJava FAILED
error: invalid source release: 21
```

Gradle이 참조하는 JDK가 여전히 17이었습니다. IntelliJ 설정 세 군데를 고쳐서 간신히 컴파일은 통과했는데, 이번엔 실행이 죽었습니다. 포트 8080이 이미 누군가에게 물려 있었습니다. 예전에 돌리던 로컬 서버가 백그라운드에서 살아 있었던 겁니다.

첫 출근 날 오전에 이러고 있다니.

한 시간 반이 지났습니다. 옆자리 동료가 의자를 돌려 말을 걸었습니다.

동료 : "아직도 안 띄워져요? 저는 출근하자마자 바로 돌렸는데요."

오픈이는 모니터에서 눈을 떼지 못했습니다.

오픈이 : "Java 버전 바꾸고, 환경 변수 잡고, 포트 충돌까지 잡았는데, 이번엔 Gradle 캐시가 꼬였대요."

동료 : "흠. 그거 저도 예전에 했었어요."

동료는 자기 화면을 돌려 보여줬습니다. 터미널 한 줄만 띄워져 있었습니다.

```
$ docker compose up
```

동료 : "저는 이걸로 띄웠어요."

docker compose up.

한 줄이었습니다. Java 버전을 맞추지도, 환경 변수를 잡지도, 포트를 정리하지도 않았는데 돌아간다고 했습니다.

오픈이 : "그게 돼요? 제 노트북에는 JDK도 Gradle도 안 깔려 있는데요."

동료 : "없어도 돼요. Docker만 있으면요."

그 말이 이상하게 들렸습니다. 내 노트북 안에 없는 프로그램이 어떻게 돌아간다는 걸까요. 팀장이 지나가다 커피잔을 내려놓으며 한마디 거들었습니다.

팀장 : "한번 생각해봐요. 왜 남의 노트북에선 되고 내 노트북에선 안 되는지. 그리고 그 차이를 **어디에 가두면** 문제가 끝날지."

차이를 가둔다. 어디에?

팀장이 돌아간 뒤에도 한참을 생각했습니다. 동료 노트북과 내 노트북의 차이는 분명했습니다. JDK 버전, Gradle 버전, 운영체제, 환경 변수, 포트 상태. 사람마다 노트북마다 다를 수 있는 것들이었습니다. 그 전부를 **한 덩어리로 묶어서** 어디에 담아두면, 누가 꺼내 쓰든 똑같이 돌아가지 않을까요.

도시락통 같은 건가.

어렸을 때 소풍 가는 아침이 떠올랐습니다. 엄마가 싸 준 도시락. 밥, 반찬, 국물 샐까 봐 꼭꼭 닫은 뚜껑. 그 도시락을 가방에 넣어 학교에 가든, 공원에 가든, 친구 집에 가든, 뚜껑만 열면 똑같은 도시락이었습니다. 어디서 먹든 안의 내용물은 바뀌지 않았습니다.

도시락통에 담아두면 어디서 열든 똑같은 밥.

그리고 그 도시락을 만드는 건 엄마의 **레시피**였습니다. 밥을 이만큼 담고, 반찬을 옆에 넣고, 계란말이는 잘라서 올리고, 뚜껑을 닫고. 레시피만 있으면 누구든 같은 도시락을 만들 수 있었습니다.

오픈이는 수첩을 폈습니다.

```
— 생각 정리 —

1. 문제의 정체
   · 내 노트북과 동료 노트북이 다르다 (JDK, Gradle, 포트, OS)
   · 그래서 같은 코드가 여기선 되고 저기선 안 된다

2. 해결 방향
   · 실행에 필요한 모든 재료를 한 덩어리로 묶는다
   · 누가 꺼내 쓰든 안은 똑같다 → 도시락통

3. 도시락을 만드는 레시피
   · 레시피를 글로 적어두면 누구든 같은 도시락을 만든다
   · 이걸 자동화한 게 Docker인 듯
```

도시락통을 만드는 레시피, 그 도시락통을 꺼내 여는 방법.

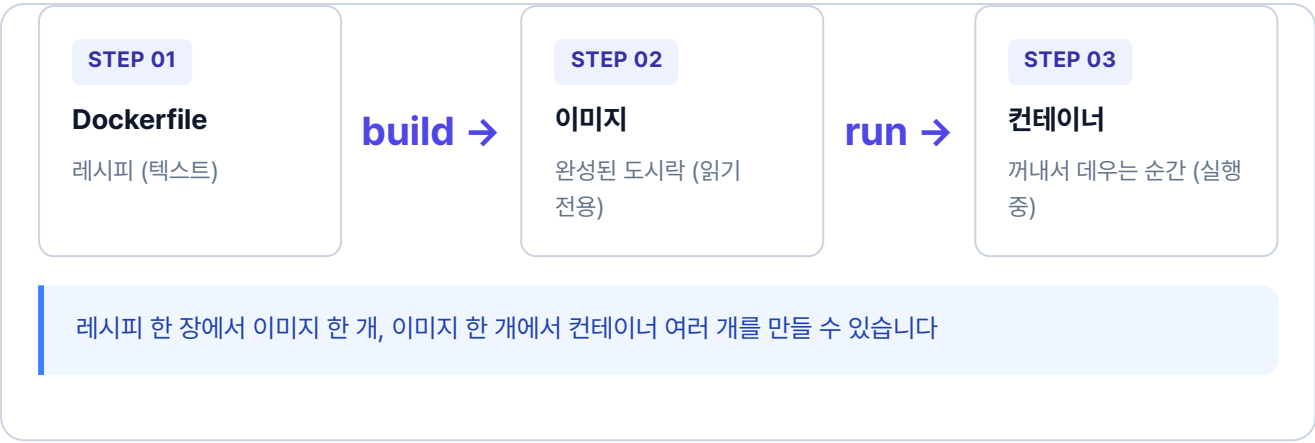
팀장이 말한 "차이를 가두는 곳"이 이 도시락통이었습니다. 내가 쓰는 JDK도, 내가 쓰는 Gradle도, 내가 쓰는 OS도, 전부 도시락통 **안**에 들어가 있다면, 바깥 노트북의 상태는 더 이상 중요하지 않았습니다. 뚜껑만 열면 됐습니다.

한번 해 보자.

1.2 도시락통과 레시피. 이미지·컨테이너·컴포즈

도시락 비유를 Docker 용어로 옮기면 이렇습니다. **레시피**가 `Dockerfile`입니다. 레시피대로 한 번 만들어 둔 **완성된 도시락**이 이미지입니다. 그 도시락을 **실제로 꺼내서 데워 먹는 순간**이 컨테이너입니다. 그리고 도시락이 여러 개 필요할 때, 어떤 도시락을 어느 자리에 어떻게 놓을지 **배치도**를 그려 둔 게 docker-compose.yml입니다.

그림 1-2. Dockerfile → 이미지 → 컨테이너 관계



> 레시피 한 장에서 이미지 한 개, 이미지 한 개에서 컨테이너 여러 개를 만들 수 있습니다

왜 이렇게 **세 단계**로 나눠 놓았을까요. 도시락으로 다시 돌아가 보겠습니다. 레시피는 종이라서 가벼우니 깃허브에 올려 공유하기 좋습니다. 완성된 도시락은 부피가 있지만, 한 번 만들어 두면 여러 사람이 가져갈 수 있어서 이미지 저장소(Docker Hub 같은 곳)에 올려 둡니다. 꺼내 먹는 행위는 그때그때 새로 해야 합니다. 같은 도시락을 두 명이 먹고 싶으면 두 개를 꺼내면 됩니다. 그래서 **이미지 한 개로 컨테이너 여러 개**를 띄울 수 있습니다.

지금은 오픈이의 서비스가 컨테이너 하나로도 충분합니다. 하지만 다음 챕터부터 Redis·S3·Elasticsearch 같은 친구들이 하나씩 붙기 시작합니다. 그때마다 컨테이너가 늘어나고, 어느 컨테이너가 먼저 켜져야 하는지, 어느 포트로 이야기해야 하는지 같은 **배치도**가 필요해집니다. 그 배치도가 바로 `docker-compose.yml`입니다.

이제 `spring-docker` 레포에 있는 구성 파일 세 개를 하나씩 읽어보겠습니다.

1.2.1 Dockerfile - 도시락 레시피

`Dockerfile`을 열겠습니다. 이 파일 한 장에 **내 서비스가 어떤 환경에서 돌아야 하는지**가 전부 적혀 있습니다.

실습 1 Dockerfile. 베이스 이미지 + 도구 설치 + 실행 스크립트

```
# 1. Eclipse Temurin 21 기반 이미지를 사용
FROM eclipse-temurin:21-jdk

# 2. 필수 패키지 설치 (git)
RUN apt-get update && apt-get install -y git

# 3. 작업 디렉터리 설정
WORKDIR /app
```

```
# 4. 로컬 entrypoint.sh 파일을 컨테이너 내부로 복사
COPY ./entrypoint.sh ./entrypoint.sh

# 5. entrypoint.sh에 실행 권한 추가
RUN chmod +x ./entrypoint.sh

# 6. 컨테이너 시작 시 entrypoint.sh 자동 실행
ENTRYPOINT ["/bin/bash", "./entrypoint.sh"]
```

여섯 줄을 요리 레시피처럼 읽으면 이렇습니다. **첫째, 빈 도마부터 시작하지 않는다.** `FROM eclipse-temurin:21-jdk`는 "Java 21이 이미 깔린 우분투 리눅스"라는 **반쯤 만들어진 도시락**을 가져오는 줄입니다. 이클립스 재단이 관리하는 공식 OpenJDK 배포판이라 믿고 쓸 수 있습니다. **둘째, 필요한 도구를 추가한다.** `RUN apt-get install -y git`은 이 도시락 위에 git을 얹는 줄입니다. 이따 서비스 코드를 git clone으로 당겨 올 예정이라서 미리 설치해 둡니다. **셋째, 작업 위치를 고정한다.** `WORKDIR /app`은 "앞으로 실행되는 모든 명령은 `/app` 폴더에서 일어난다"고 선언하는 줄입니다. **넷째·다섯째, 실행 스크립트를 넣고 권한을 준다.** `entrypoint.sh`라는 파일을 컨테이너 안으로 복사하고, 실행 가능하게 만듭니다. **여섯째, 컨테이너가 켜지면 이 스크립트를 돌린다.** `ENTRYPOINT`는 도시락 뚜껑을 열었을 때 **자동으로 돌아가는 행동**을 정하는 줄입니다.

여기서 한 가지 헷갈릴 수 있는 지점이 있습니다. Dockerfile에는 Spring 서비스 코드가 어디에도 안 보입니다. 그럴 수밖에 없습니다. 실제로 서비스 코드를 가져와서 빌드하는 일은 **이미지를 굽는 시점이 아니라, 컨테이너가 켜지는 시점**에 일어나기 때문입니다. 그 일을 맡는 스크립트가 바로 `entrypoint.sh`입니다.

1.2.2 entrypoint.sh - 뚜껑을 열었을 때 해야 할 일

실습 2 entrypoint.sh. git clone → 빌드 → 실행

```
# 1. Git 리포지토리 주소 설정
git clone https://github.com/kjh5848/spring-app

# 2. 프로젝트 디렉터리로 이동
cd spring-app

# 3. gradlew 실행 권한 부여
chmod +x ./gradlew

# 4. Gradle 빌드 (테스트 생략)
./gradlew clean build
```

```
# 5. jar 파일 실행 권한 부여
chmod +x build/libs/*.jar

# 6. Spring Boot 실행
java -jar build/libs/*.jar
```

컨테이너가 처음 켜지면 `spring-app` 저장소를 깃허브에서 당겨 옵니다. 프로젝트 폴더로 들어가서, Gradle 래퍼에 실행 권한을 주고, `./gradlew clean build`로 빌드를 돌립니다. 빌드가 끝나면 `build/libs` 아래에 jar 파일이 하나 생깁니다. 마지막 줄에서 그 jar를 `java -jar`로 실행합니다.

이 여섯 줄이 **호스트 노트북에서 오픈이가 한 시간 반 동안 고생했던 모든 과정**입니다. Java 설치, 환경 변수 설정, 빌드, 실행. 전부 컨테이너 **안**에서 일어납니다. 바깥에서 오픈이가 할 일은 `docker compose up` 한 줄뿐입니다.

> **운영에서는 이렇게 하지 않습니다**
> 실무 서비스에서는 보통 깃을 컨테이너 안에서 돌리지 않습니다. 대신 CI 파이프라인이 미리 빌드해 둔 jar 파일을 이미지에 `COPY`로 넣어 두거나, **멀티 스테이지 빌드**(`FROM ... AS build` → `FROM ... AS runtime`)로 빌드 환경과 실행 환경을 분리합니다. 이번 챕터는 "한 줄 명령으로 뜨는 경험"이 우선이라 가장 단순한 방식을 썼습니다.

1.2.3 docker-compose.yml - 배치도

마지막 한 장입니다.

실습 3 docker-compose.yml. 컨테이너를 어떻게 띄울지 정의

```
services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: spring-docker
    ports:
      - "8080:8080"
```

`services:` 아래에 `app`이라는 이름의 컨테이너 정의가 하나 있습니다. `build`는 "이미지를 미리 만들어 둔 게 없으니, 현재 폴더(`context: .`)의 `Dockerfile`로 지금 구워라"는 지시입니다. `container_name`은 컨테이너에 붙일 이름표입니다. 기본값을 쓰면 `spring-docker_app_1` 같은

자동 생성 이름이 붙지만, 여기서는 보기 좋게 `spring-docker`로 고정했습니다. 마지막 `ports: "8080:8080"`은 "호스트의 8080 포트로 들어오는 요청을 컨테이너 안의 8080 포트로 연결해 달라"는 줄입니다. 앞 숫자가 호스트, 뒤 숫자가 컨테이너입니다.

항목	값	역할
`services.app`	—	컴포즈가 관리할 컨테이너 한 개의 이름
`build.context`	`.`	이미지를 구울 때 참조할 디렉토리
`build.dockerfile`	`Dockerfile`	레시피 파일 이름
`container_name`	`spring-docker`	실행 후 확인할 때 쓸 컨테이너 별명
`ports`	`8080:8080`	호스트 포트 : 컨테이너 포트

서비스가 하나뿐이라 배치도가 단출합니다. 챕터 3에서 Redis가 들어오면 `services:` 아래에 `redis:` 블록이 한 덩어리 더 붙을 예정입니다. 그때는 `depends_on`(어느 컨테이너가 먼저 켜져야 하는지)과 `networks`(컨테이너들끼리 어떤 이름으로 통신할지)도 함께 등장합니다. 지금은 "배치도 위에 도시락 한 칸이 올라가 있다"는 그림만 머리에 넣어 두면 충분합니다.

1.3 한 줄로 서버 띄우기. docker compose up

구성 파일 세 개가 머릿속에 들어왔으니, 이제 실행해 볼 차례입니다. 먼저 Docker Desktop이 켜져 있는지 확인합니다.

그림 1-3. Docker Desktop이 실행 중이면 상단에 고래 아이콘이 초록색으로 떠 있습니다

그다음 `spring-docker` 폴더에서 터미널을 열고 한 줄을 입력합니다.

터미널 실험 1-1 실행. docker compose up

```
docker compose up
```

처음 돌리면 시간이 꽤 걸립니다. 순서대로 이런 일이 일어납니다.

1. `eclipse-temurin:21-jdk` 이미지를 Docker Hub에서 내려받는다 (약 400MB)
2. 그 이미지 위에 git을 설치한다
3. `entrypoint.sh`를 복사하고 권한을 준다
4. 여기까지가 **이미지 굽기**. 이 단계가 끝나면 `spring-docker-app` 이라는 이름의 이미지가 생긴다
5. 이미지로 컨테이너를 하나 띄운다 (`spring-docker`)
6. 컨테이너 안에서 `entrypoint.sh`가 돌기 시작한다. `spring-app` 저장소 클론 → Gradle 빌드 → jar 실행
7. 마지막 줄에 `Started SpringDokerApplication in 3.1 seconds` 같은 로그가 찍힌다

[CAPTURE NEEDED: spring-docker 폴더에서 docker compose up 실행. 이미지 pull → build → 컨테이너 기동 → Spring Boot 배너 + "Tomcat started on port(s): 8080" 로그까지 보이는 터미널]

그림 1-4. `docker compose up` 실행 직후. 이미지 빌드와 Gradle 빌드 로그가 연달아 흐릅니다

첫 실행은 5~10분쯤 걸릴 수 있습니다. 두 번째부터는 이미지가 캐시되어 몇 초면 끝납니다. 마지막에 `Tomcat started on port(s): 8080`이 찍혔다면 성공입니다.

Docker Desktop 창으로 가 보면 **Containers** 탭에 `spring-docker` 한 줄이 초록색 Running으로 떠 있습니다.

그림 1-5. Docker Desktop Containers 탭. `spring-docker` 컨테이너가 Running 상태로 보입니다

컨테이너가 돌고 있다.

오픈이는 의자에 등을 기대고 화면을 바라봤습니다. 한 시간 반을 붙잡고 있던 JDK 설치, 환경 변수, 포트 충돌이 전부 사라졌습니다. 정확히 말하면 **사라진 게 아니라, 컨테이너 안으로 들어간 것**이었습니다. 밖에서는 더 이상 보이지 않을 뿐.

1.4 결과 확인과 컨테이너 정리

서버가 떴다면 브라우저 주소창에 `localhost:8080`을 입력합니다.

그림 1-6. 브라우저에 `hello world!.`가 찍혔습니다. health check 성공

`hello world!.` 한 줄. `spring-app` 저장소의 `SpringDokerController`에 있던 `@GetMapping("/")` 핸들러가 돌아왔다는 뜻입니다. 이제 컨테이너를 내려 보겠습니다. 새 터미널을 열고(서버 로그가 돌고 있는 터미널은 그대로 두고) 같은 폴더에서 입력합니다.

터미널 실험 1-2 실행. 컨테이너 정리

```
docker compose down
```

이 한 줄이 여러 가지를 정리합니다.

그림 1-7. `docker compose down`이 정리하는 것

컨테이너	실행 중이던 `spring-docker`를 정지하고 삭제
네트워크	컴포즈가 만들어 두었던 `spring-docker_default` 네트워크 삭제
이미지	그대로 남음. 다음 `up`에서 재사용
볼륨	명시하지 않았으니 해당 없음 (챕터 3에서 등장)

> 컨테이너·네트워크는 내려가지만 이미지는 남아서, 다음 `up`은 빠릅니다

그림 1-8. `docker compose down` 실행 결과. 컨테이너와 네트워크가 Removed로 표시됩니다

Docker Desktop Containers 탭을 다시 열면 `spring-docker` 항목이 사라진 걸 확인할 수 있습니다. 이미지 탭으로 가면 `spring-docker-app`과 `eclipse-temurin` 이미지는 그대로 남아 있습니다. 다음에 `docker compose up`을 다시 돌리면 이 이미지들을 재사용해서 몇 초 만에 서버가 뜹니다.

compose up·down 실험에서 생각해 볼 것들

- **첫 실행이 느린 이유**: 베이스 이미지 다운로드 + git clone + Gradle 빌드가 전부 **처음 한 번**은 네트워크로 돌아갑니다. 두 번째부터는 이미지 레이어가 캐시되고, `entrypoint.sh` 안의 git clone·빌드 단계만 새로 돕니다
- **컨테이너 안의 변경은 사라진다**: `docker compose down` 한 번이면 `/app` 폴더에 만들어 둔 jar, 캐시, 로그가 전부 지워집니다. **영구 저장**이 필요한 데이터는 볼륨에 빼 둬야 합니다. 챕터 3에서 Redis를 붙일 때 처음 다뤄 보겠습니다
- `docker compose up -d`: 끝에 `-d` (detach)를 붙이면 백그라운드에서 돕니다. 터미널을 다른 일로 쓰고 싶다면 `-d` 로 올리고, 로그는 `docker compose logs -f` 로 따라가면 됩니다
- **포트 매핑의 방향**: `8080:8080` 중 앞이 호스트, 뒤가 컨테이너입니다. 내 노트북 8080이 이미 쓰이고 있다면 `9090:8080` 으로 올려서 브라우저는 `localhost:9090` 으로 접속하면 됩니다. 컨테이너 안쪽은 바뀌지 않습니다

한 줄이었다.

오픈이는 팀장에게 메신저로 스크린샷을 한 장 보냈습니다. `localhost:8080` 에 떠 있는 `hello world!.` 였습니다.

오픈이 : "환경 세팅 끝냈습니다."

팀장 : "오전 안에 끝냈네요. 내일부터는 이 서비스에 기능을 하나씩 얹을 거예요. 다음 주에는 카카오 로그인부터 붙일 건데요."

로그인.

외부 서비스가 하나 끼어드는 첫 장면이었습니다. 내 서버 하나만 있을 때는 컨테이너 한 칸이면 충분했습니다. 그런데 로그인을 외부에 맡기면 **내 서비스 바깥의 누군가**와 이야기해야 합니다. 그 누군가는 어떻게 내 서비스를 알아볼까요. 매번 아이디와 비밀번호를 주고받는 걸까요. 아니면 어떤 다른 방식이 있을까요.

오픈이는 수첩 한 쪽 귀퉁이에 물음표 하나를 그려 두고 노트북을 닫았습니다.

용어 정리

이야기 속 표현	진짜 용어	정식 정의
도시락 레시피	Dockerfile	이미지 빌드 절차를 명령어 단위로 적어 둔 텍스트 파일
완성된 도시락	이미지(Image)	실행에 필요한 파일 시스템·라이브러리·설정을 한 덩어리로 묶은 읽기 전용 스냅샷
꺼내서 데우는 순간	컨테이너(Container)	이미지를 기반으로 실행 중인 격리된 프로세스 환경. 파일 시스템·네트워크·프로세스가 호스트와 분리됨
배치도	docker-compose.yml	여러 컨테이너의 이미지·포트·네트워크·볼륨 구성을 한 파일로 선언하는 YAML. Compose v2부터 `docker compose` (띄어쓰기) 명령으로 실행
컨테이너 시작 동작	ENTRYPOINT / entrypoint.sh	컨테이너가 시작될 때 자동으로 실행되는 명령어. Dockerfile의 `ENTRYPOINT`로 지정
반쯤 만들어진 도시락	베이스 이미지(Base Image)	`FROM` 구문으로 가져오는 출발점 이미지. 예: `eclipse-temurin:21-jdk`는 Java 21이 설치된 리눅스
호스트 포트 : 컨테이너 포트	포트 매핑(Port Mapping)	호스트의 특정 포트로 들어온 트래픽을 컨테이너 안의 포트로 연결. `ports: "8080:8080"` 형식

이것만은 기억하자

- **환경 차이는 컨테이너 안에 가둔다.** Java 버전·환경 변수·운영체제가 달라도 도시락통 안만 똑같으면 결과가 같다
- **구성 파일 세 장이면 서비스 한 개가 뜬다.** `Dockerfile`(레시피) → 이미지 → 컨테이너, 그리고 배치도 한 장(`docker-compose.yml`)
- `docker compose up` / `docker compose down`**으로 켜고 끈다.** 한 줄 명령어 두 개가 오늘 모든 실습의 입구와 출구다
- **다음 챕터에서는** 이 서비스에 **카카오 로그인**을 붙입니다. 외부 인증 서버와 대화할 때, 매번 비밀번호를 주고받지 않고 **통행증 하나**로 끝내는 방법 (OAuth2.0 & OIDC)을 만나 보겠습니다